

Паттерны проектирования

Лекции: 32 часа. Лабораторные работы: 32 часа. Зачет.

Содержание: 22 паттерна.

Инструменты: C#, JavaScript, <https://www.draw.io>.

Лекции, задания на лабораторные работы:

Для_студентов_ФИТ_БГТУ > ПРЕПОДАВАТЕЛИ > Смелов > II КУРС > ПП			
Создать ▾	Загрузить ▾	Действие ▾	Инструменты ▾
Название ^	Ра...	Тип файла	Дата изменения
Лабораторные работы	Папка		2023-02-07 02:42:04
Лекции	Папка		2023-02-08 02:36:00

Литература:

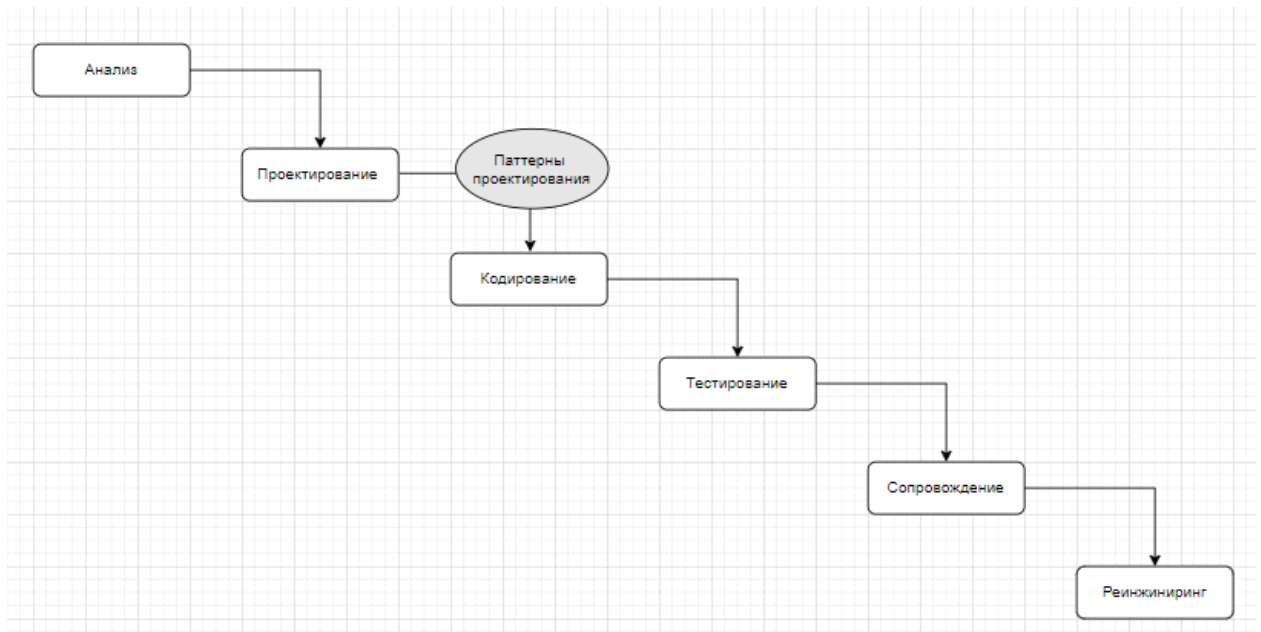
Для_студентов_ФИТ_БГТУ > ЛИТЕРАТУРА			
Создать ▾	Загрузить ▾	Действие ▾	Инструменты ▾
Название ^	Ра...	Тип файла	Настро
Патерны_ПроектированиеПО	Папка		

Введение

1. **Паттерн:** pattern – узор, шаблон, схема.
2. **Паттерн проектирования:** схема решения часто встречаемой задачи, прием программирования, **computer science:** технология программирования, паттерн=шаблон, архитектор: применить паттерн, не библиотека, не фреймворк, не алгоритм, достоинства: проверенные решения, стандартный код, программистский словарь.
3. **Паттерн проектирования:** Кристофер Александер для проектирования человеческого окружения (городская архитектура, ландшафт, мебель...), Эрих Гамме, Ричард

Хелм, Ральф Джонсон, Джон Влиссидес «Design Patterns: Elements of Reusable Object-Oriented Software», gang of four (GOF) – банда четырех.

4. Решение о применении паттерна



5. **Реализация паттерна:** разработка программного кода в соответствии с паттерном; 2 реализации одного и того же паттерна могут сильно отличаться.

6. **Классификация:** порождающие (гибкие методы создания объектов), структурные (связи между объектами), поведенческие (коммуникация между объектами).

7. **Классификация: идиомы** – низкоуровневые (зависят от языка программирования); **архитектурные** – самые высокоуровневые.

8. **Повторное использования кода:** реализованные в рамках системы программирования (классы, функции, библиотеки, контейнеры (коллекции, итераторы)), паттерны программирования, фреймворки (паттерн, поддерживаемый технологией, библиотекой, инструментарием).

9. Описание паттерна:

- описание проблемы;
- мотивация к применению;
- UML-диаграмма;
- структуры применяемых классов;
- пример на одном из языков программирования;

- особенности реализации;
- связи с другими паттернами.

10. **Фундамент паттернов проектирования:** объектно-ориентированное программирование. Наследование, инкапсуляцию и полиморфизм можно считать паттернами встроенными в систему ООП-программирования.

Объектно-ориентированное программирование (повторение)

11. **ООП:** распространение методов системного анализа на процесс проектирования программного обеспечения.

12. **ООП:** упростить взаимодействие **бизнес аналитик-проектировщик – разработчик**.

13. **ООП:** парадигмы: абстракция, инкапсуляция, полиморфизм.

14. **Объекты и классы:** класс: тип, структура объекта, объект: экземпляр класса, объект имеет тип. Класс – тип элементов системы (результат системного анализа); класс/объект: состояние, поведение

Абстракция

15. **Абстракция:** моделирование системы, классы – типы объектов, объекты – экземпляры типов, классы/объекты отражают реальный объект только с точки зрения наблюдателя.

Инкапсуляция

16. **Инкапсуляция:** способность объектов скрывать часть своего состояния и поведения.

17. **Инкапсуляция/проблема:** как сделать так, чтобы пользователь класса/объекта имел доступ только к тем свойствам, полям и методам, которые необходимы пользователям библиотеки классов?

18. **Инкапсуляция/мотивация:** контроль доступа к свойствам, состоянию и методам объекта; степень свободы для разработчика кода: пользовательский код зависит только от доступных ему членов класса.

19. **Инкапсуляция/UML:**

+University	+Faculty
<<getter>>+id:int	<<getter>>+id:int
<<setter>>-id:int	<<setter>>-id:int
<<getter>>+name:string	<<getter>>+name:string
<<setter>>#name:string	<<setter>>#name:string
<<getter>>+shortName:type	<<getter>>+shortName:type
<<setter>>#shortName:type	<<setter>>#shortName:type
<<getter>>+address:string	<<getter>>+address:string
<<setter>>#address:string	<<setter>>#address:string
#faculties:List<Faculty>	#departments:List<Department>
-timeStamp: Datetime	-timeStamp: Datetime
<<constructor>>+University()	<<constructor>>+Faculty()
<<constructor>>+University(University)	<<constructor>>+Faculty(Faculty)
<<constructor>>+University(string name, string shortName, string address)	<<constructor>>+Faculty(string name, string shortName, string address)
+addFaculty(Faculty):int	+addDepartment(Department):int
+delFaculty(int):bool	+delDepartment(int):bool
+updFaculty(Faculty):bool	+updDepartment(Department):bool
-verFaculty(int):bool	-verDepartment(int):bool
+getFaculties():List<Faculty>	+getDepartments():List<Department>

20. Инкапсуляция/С#

```
public partial class University
{
    private int _id = 0;
    public int id
    {
        get{return this._id;}
        private set{ this._id = value;}
    }
    private string _name;
    public string name
    {
        get { return this._name; }
        protected set { this._name = value; }
    }
    private string _shortName;
    public string shortName
    {
        get { return this._shortName; }
        protected set { this._shortName = value; }
    }
    private string _address;
    public string address
    {
        get { return this._address; }
        protected set { this._address = value; }
    }
    protected List<Faculty> faculties;
    private DateTime timestamp;
    public University()
    {
        // значения по умолчанию
    }
    public University(string name, string shortName, string address)
    {
        // значения по умолчанию
    }
}
```

```

public University(University university)
{
    // значения по умолчанию
}
private bool verFaculty(Faculty faculty)
{
    bool rc = true;
    return rc;
}
public int addFaculty(Faculty faculty)
{
    int rc = 1;
    if (verFaculty(faculty))
    {
        // добавить в БД, сгенерировать rc
    }
    return rc;
}
public bool updFaculty(Faculty faculty)
{
    bool rc = true;
    if (verFaculty(faculty))
    {
        // изменить в БД, сгенерировать rc
    }
    return rc;
}
public bool delFaculty(int id)
{
    bool rc = true;
    // удалить из БД, сгенерировать rc
    return rc;
}
public List<Faculty> getFaculties()
{
    // прочитывать из БД, записать в this.faculties
    return this.faculties;
}
}

```

```

public class Faculty
{
    private int _id = 0;
    public int id
    {
        get { return this._id; }
        private set { this._id = value; }
    }
    private string _name;
    public string name
    {
        get { return this._name; }
        protected set { this._name = value; }
    }
    private string _shortName;
    public string shortName
    {
        get { return this._shortName; }
        protected set { this._shortName = value; }
    }
    private string _address;
    public string address
    {
        get { return this._address; }
        protected set { this._address = value; }
    }
    protected List<Department> departments;
    private DateTime timestamp;
    public Faculty()
    {
        // значения по умолчанию
    }
    public Faculty(string name, string shortName, string address)
    {
        // значения по умолчанию
    }
}

```

```

public Faculty(Faculty faculty)
{
    // значения по умолчанию
}
private bool verDepartment(Department faculty)
{
    bool rc = true;
    return rc;
}
public int addDepartment(Department department)
{
    int rc = 1;
    if (verDepartment(department))
    {
        // добавить в БД, сгенерировать rc
    }
    return rc;
}
public bool updDepartment(Department department)
{
    bool rc = true;
    if (verDepartment(department))
    {
        // изменить в БД, сгенерировать rc
    }
    return rc;
}
public bool delDepartment(int id)
{
    bool rc = true;
    // удалить из БД, сгенерировать rc
    return rc;
}
public List<Department> getDepartments()
{
    // прочитав из БД, записать в this.faculties
    return this.departments;
}
}

```

21. **Инкапсуляция/JS:**

22. **Инкапсуляция/особенности:** нет, это базовый паттерн

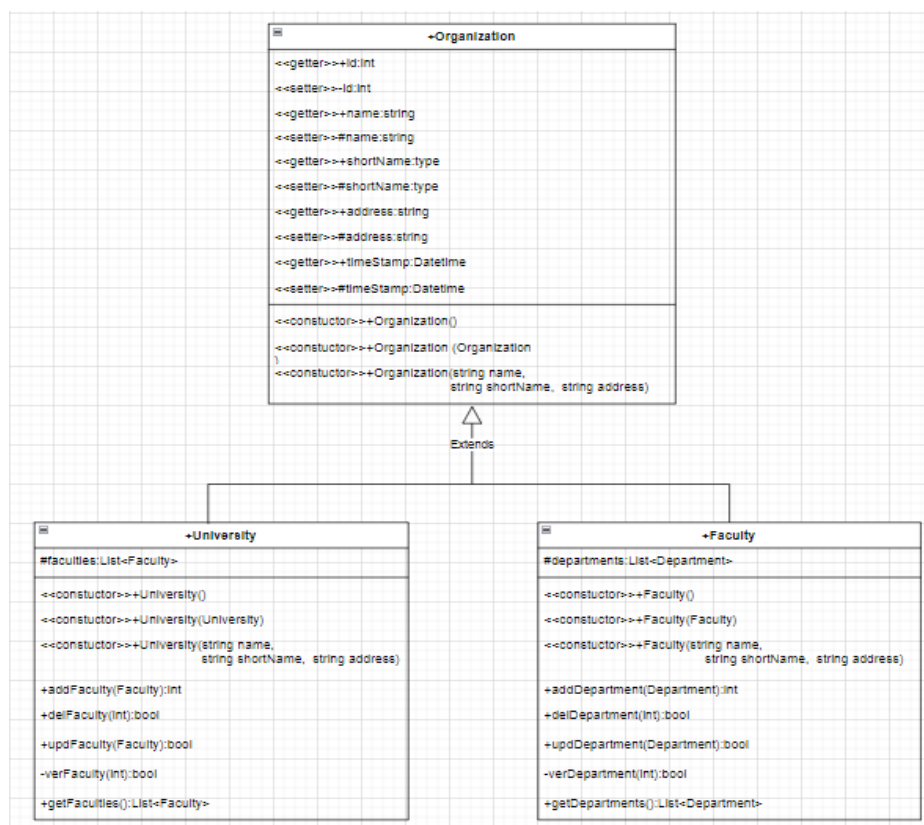
23. **Инкапсуляция/связь:** нет, это базовый паттерн

24. **Наследование:** повторное использование кода, позволяющее на основе одного класса, создать другой; принцип *ЕСТЬ*. Виртуальные функции, абстрактные функции, абстрактный класс.

25. **Наследование/проблема:** много классов с одинаковыми свойствами и методами и между ними можно установить отношение *ЕСТЬ*.

26. **Наследование/мотивация:** сокращение кода.

27. **Наследование/UML:**



28. Наследование/C#

```

public class Organization
{
    private int _id = 0;
    public int id
    {
        get { return this._id; }
        private set { this._id = value; }
    }
    private string _name;
    public string name
    {
        get { return this._name; }
        protected set { this._name = value; }
    }
    private string _shortName;
    public string shortName
    {
        get { return this._shortName; }
        protected set { this._shortName = value; }
    }
    private string _address;
    public string address
    {
        get { return this._address; }
        protected set { this._address = value; }
    }
    public Organization()
    {
        // значения по умолчанию
    }
    public Organization(string name, string shortName, string address)
    {
        // значения по умолчанию
    }
    public Organization(Organization university)
    {
        // значения по умолчанию
    }
}
  
```

```

class University:Organization
{
    protected List<Faculty> faculties;
    public University()
    {
        // значения по умолчанию
    }
    public University(string name, string shortName, string address):base(name,shortName,address)
    {
        // значения по умолчанию
    }
    public University(University university):base(university)
    {
        // копирующий конструктор
    }
    private bool verFaculty(Faculty faculty)
    {
        ..
        bool rc = true;
        return rc;
    }
    public int addFaculty(Faculty faculty)
    {
        {
            int rc = 1;
            if (verFaculty(faculty))
            {
                // добавить в БД, сгенерировать rc
            }
            return rc;
        }
    }
    public bool updFaculty(Faculty faculty)
    {
        {
            bool rc = true;
            if (verFaculty(faculty))
            {
                // изменить в БД, сгенерировать rc
            }
            return rc;
        }
    }
    public bool delFaculty(int id)
    {
        return true;
    }
    public List<Faculty> getFaculties()
    {
        // прочитайте из БД, записать в this.faculties
        return this.faculties;
    }
}

```

```

public class Faculty: Organization
{
    protected List<Department> departments;
    public Faculty()
    {
        // значения по умолчанию
    }
    public Faculty(string name, string shortName, string address):base(name,shortName, address)
    {
        // установить значения по умолчанию
    }
    public Faculty(Faculty faculty) : base(faculty)
    {
        // копирующий конструктор
    }
    private bool verDepartment(Department department)
    {
        return true;
    }
    public int addDepartment(Department department)
    {
        {
            int rc = 1;
            if (verDepartment(department))
            {
                // добавить в БД, сгенерировать rc
            }
            return rc;
        }
    }
    public bool updDepartment(Department department)
    {
        {
            bool rc = true;
            if (verDepartment(department))
            {
                // изменить в БД, сгенерировать rc
            }
            return rc;
        }
    }
    public bool delDepartment(int id)
    {
        return true;
    }
    public List<Department> getDepartments()
    {
        {
            return this.departments;
        }
    }
}

```


29. **Наследование/JS:**

30. **Наследование/особенности:** нет, это базовый паттерн

31. **Наследование/связь:** нет, это базовый паттерн

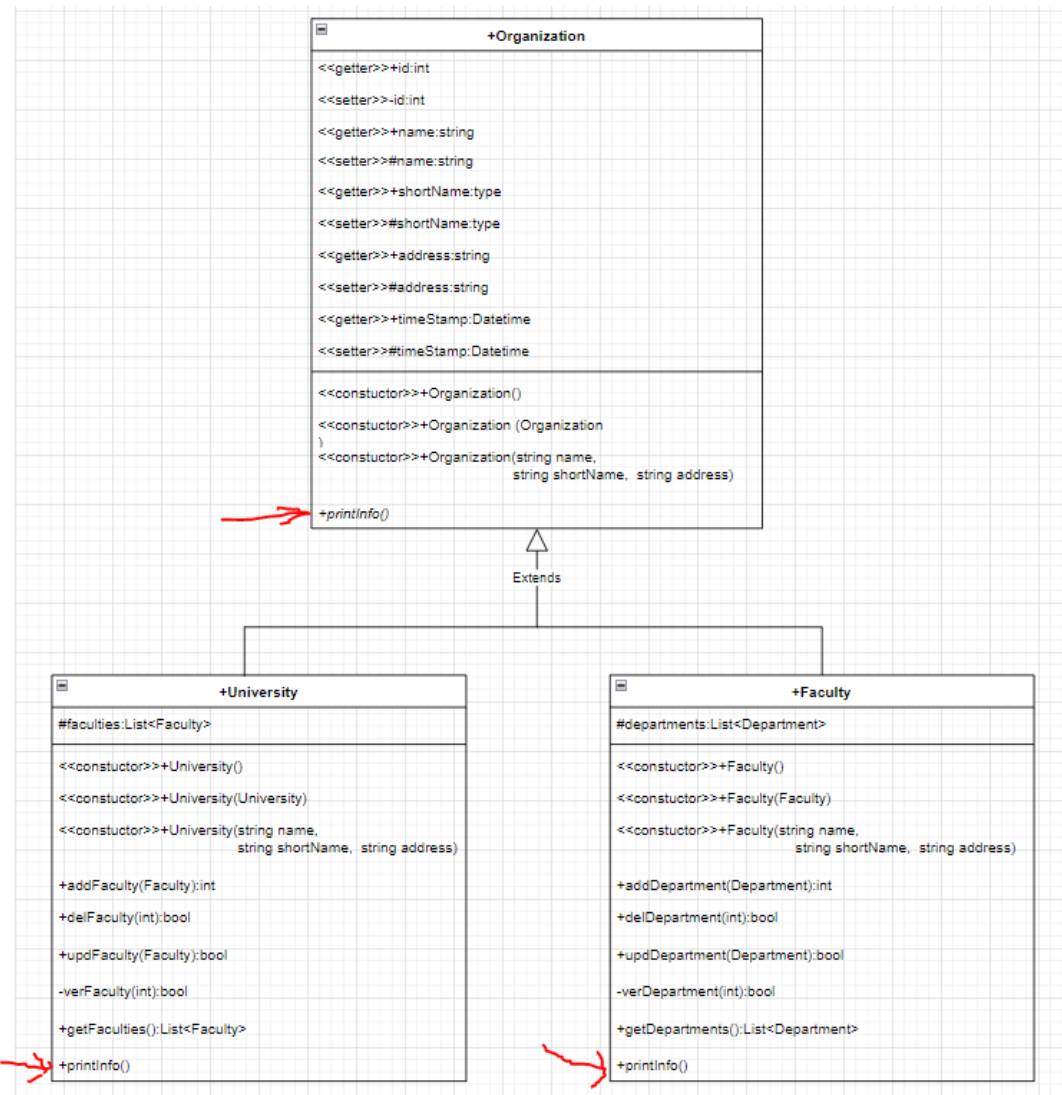
Полиморфизм

32. **Полиморфизм:** способность объекта изменять свое поведение в зависимости от контекста его использования. Простейшие: перезагружаемые методы, параметры по умолчанию, несколько конструкторов. Сложнее: вытеснение виртуальных функций в базовом классе и иерархии классов.

33. **Полиморфизм/проблема:** как сделать так, чтобы базовый объект (при наследовании) изменял свое поведения в зависимости от производного объекта.

34. **Полиморфизм/мотивация:** упростить (за счет возможностей компилятора) применение класса, .

35. **Полиморфизм/UML:**



36. Полиморфизм/С#

```

public class Organization
{
    private int _id = 0;
    public int id...
    private string _name;
    public string name...
    private string _shortName;
    public string shortName...
    private string _address;
    public string address...
    public Organization()...
    public Organization(string name, string shortName, string address)...
    public Organization(Organization organization)...
    virtual public string PrintInfo()
    {
        return string.Format("Организация: {0} \nАдрес : {1} \n", this.name, this.address);
    }
}
  
```

```

public class University : Organization
{
    protected List<Faculty> faculties;
    public University()...
    public University(string name, string shortName, string address) ...
    public University(University university) ...
    private bool verFaculty(Faculty faculty)...
    public int addFaculty(Faculty faculty)...
    public bool updFaculty(Faculty faculty)...
    public bool delFaculty(int id)...
    public List<Faculty> getFaculties()...
    public override string PrintInfo()
    {
        return string.Format("Университет: {0} \nАдрес      : {1} \n", this.name, this.address);
    }
}

```

```

public class Faculty : Organization
{
    protected List<Department> departments;
    public Faculty()...
    public Faculty(string name, string shortName, string address) ...
    public Faculty(Faculty faculty) ...
    private bool verDepartment(Department department)...
    public int addDepartment(Department department)...
    public bool updDepartment(Department department)...
    public bool delDepartment(int id)...
    public List<Department> getDepartments()...
    public override string PrintInfo()
    {
        return string.Format("Факультет: {0} \nАдрес      : {1} \n", this.name, this.address);
    }
}

```

```

static void Main(string[] args)
{
    University bstu = new University("Белорусский государственный технологический университет",
                                     "БГТУ",
                                     "г. Минск, Свердлова, 13А");

    Faculty fit = new Faculty("Информационных технологий", "ИТ", "г. Минск, Свердлова, 13А, 101-4");
    Faculty fie = new Faculty("Инженерно-экономический", "ИЭ", "г. Минск, Свердлова, 13А, 304-4");
    Faculty flh = new Faculty("Лесохозяйственный", "ЛХ", "г. Минск, Свердлова, 13А, 210-1");

    bstu.addFaculty(fit);
    bstu.addFaculty(fie);
    bstu.addFaculty(flh);

    Organization org_bstu = bstu;
    Organization org_fit = fit;
    Organization org     = new Organization("Стадион Динамо", "Динамо", "г. Минск, ул. Ульяновская, 8");
    Console.WriteLine(org_bstu.PrintInfo());
    Console.WriteLine(org_fit.PrintInfo());
    Console.WriteLine(org.PrintInfo());
}

```

```
Консоль отладки Microsoft Visual Studio
Университет: Белорусский государственный технологический университет
Адрес      : г. Минск, Свердлова, 13А

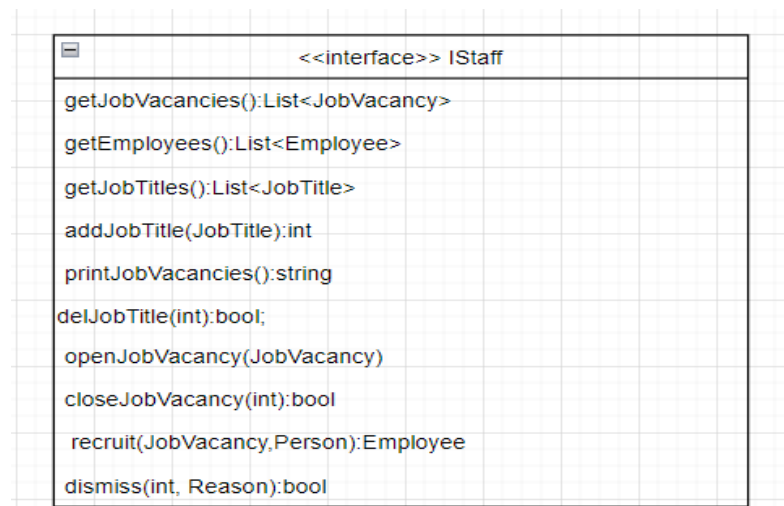
Факультет: Информационных технологий
Адрес      : г. Минск, Свердлова, 13А, 101-4

Организация: Стадион Динамо
Адрес      : г. Минск, ул. Ульяновская, 8
```

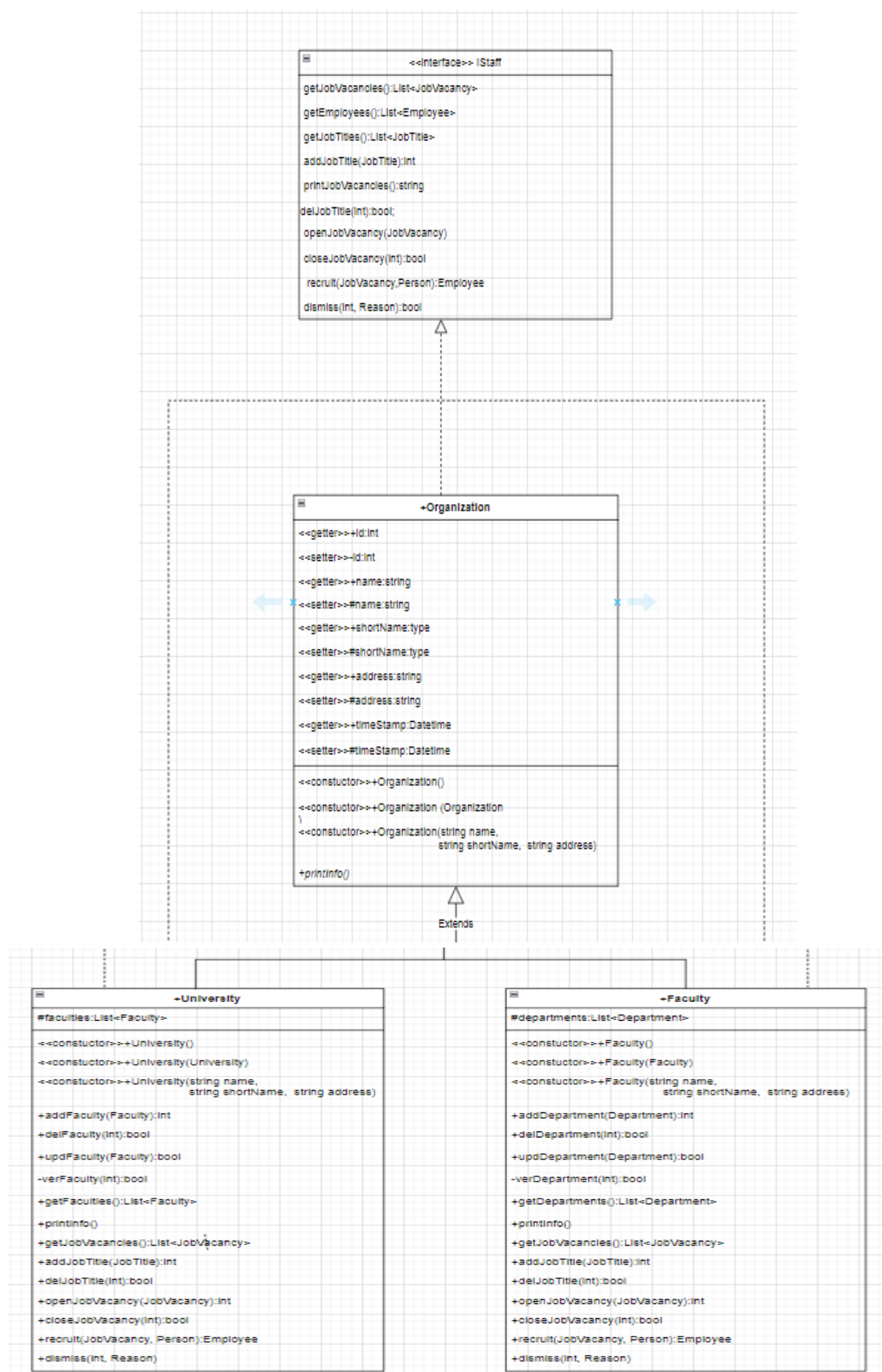
37. **Полиморфизм/особенности**: нет, это базовый паттерн

38. **Полиморфизм/связь**: нет, это базовый паттерн

39. **Интерфейс**: перечень сигнатур методов, предназначенных для реализации; контракт с фреймворком; спецификация фреймворка.



40. **Интерфейс**:



41. Интерфейс :

```

public interface IStaff
{
    List<JobVacancy> getJobVacancies();           // список вакансий
    List<Employee>   getEmployees();             // список сотрудников
    List<JobTitle>   getJobTitles();             // список должностей
    string printJobVacancies();
    int   addJobTitle(JobTitle jobTitle);
    bool  delJobTitle(int id);
    int   openJobVacancy(JobVacancy jobVacancy);
    bool  closeJobVacancy(int id);
    Employee recruit(JobVacancy jobVacancy, Person person);
    bool  dismiss(int id, Reason reason);
}

```

```

public class Organization: IStaff
{
    private int _id = 0;
    public int id...
    private string _name;
    public string name...
    private string _shortName;
    public string shortName...
    private string _address;
    public string address...
    public Organization()...
    public Organization(string name, string shortName, string address)...
    public Organization(Organization organization)...
    virtual public string PrintInfo() ...
    List<JobVacancy> IStaff.getJobVacancies()...
    List<Employee> IStaff.getEmployees()...
    List<JobTitle> IStaff.getJobTitles()...
    int IStaff.addJobTitle(JobTitle jobTitle)...
    bool IStaff.delJobTitle(int id)...
    int IStaff.openJobVacancy(JobVacancy jobVacancy)...
    bool IStaff.closeJobVacancy(int id)...
    Employee IStaff.recruit(JobVacancy jobVacancy, Person person)...
    bool IStaff.dismiss(int id, Reason reason)...
    string IStaff.printJobVacancies()...
}

```

```

public class University : Organization, IStaff
{
    protected List<Faculty> faculties;
    public University()...
    public University(string name, string shortName, string address) ...
    public University(University university) ...
    private bool verFaculty(Faculty faculty)...
    public int addFaculty(Faculty faculty)...
    public bool updFaculty(Faculty faculty)...
    public bool delFaculty(int id)...
    public List<Faculty> getFaculties()...
    List<JobVacancy> IStaff.getJobVacancies()...
    string IStaff.printJobVacancies()...
    public override string PrintInfo()...
}

```

```

public class Faculty : Organization, IStaff
{
    protected List<Department> departments;
    public Faculty()...
    public Faculty(string name, string shortName, string address) ...
    public Faculty(Faculty faculty) ...
    private bool verDepartment(Department department)...
    public int addDepartment(Department department)...
    public bool updDepartment(Department department)...
    public bool delDepartment(int id)...
    public List<Department> getDepartments()...
    string IStaff.printJobVacancies()...
    public override string PrintInfo()...
    List<JobVacancy> IStaff.getJobVacancies()...
}

```

```

static void Main(string[] args)
{
    University bstu = new University("Белорусский государственный технологический университет",
                                     "БГТУ",
                                     "г. Минск, Свердлова, 13А");
    Faculty fit = new Faculty("Информационных технологий", "ИТ", "г. Минск, Свердлова, 13А, 101-4");
    Faculty fie = new Faculty("Инженерно-экономический", "ИЭ", "г. Минск, Свердлова, 13А, 304-4");
    Faculty flh = new Faculty("Лесохозяйственный", "ЛХ", "г. Минск, Свердлова, 13А, 210-1");

    bstu.addFaculty(fit);
    bstu.addFaculty(fie);
    bstu.addFaculty(flh);

    Organization org_bstu = bstu;
    Organization org_fit = fit;
    Organization org = new Organization("Стадион Динамо", "Динамо", "г. Минск, ул. Ульяновская, 8");

    IStaff x = new Faculty("Принттехнологий и медиакоммуникаций", "ПМ", "г. Минск, Свердлова, 13А, 150-4");

    Console.WriteLine(org_bstu.PrintInfo());
    Console.WriteLine(org_fit.PrintInfo());
    Console.WriteLine(org.PrintInfo());
    Console.WriteLine(((IStaff)org).printJobVacancies());
    Console.WriteLine(((IStaff)bstu).printJobVacancies());
    Console.WriteLine(((IStaff)fit).printJobVacancies());
    Console.WriteLine(((IStaff)fie).printJobVacancies());
    Console.WriteLine(((IStaff)org_fit).printJobVacancies());
    Console.WriteLine(x.printJobVacancies());
}

```

42. Интерфейс: контракт с фреймворком.

```
public static class Process
{
    static public List<JobVacancy> mergeJobVacancies(params IStaff[] staff)
    {
        List<JobVacancy> rc = new List<JobVacancy>();
        foreach (IStaff jv in staff)
        {
            rc.AddRange(jv.getJobVacancies());
        }
        return rc;
    }

    static public string printJobVacancies(List<JobVacancy> list)
    {
        string rc = "";
        foreach (var jv in list)
        {
            rc += string.Format("{0} - {1}\n", jv.ID, jv.Name);
        }
        return rc;
    }
}
```

```
static void Main(string[] args)
{
    University bstu = new University("Белорусский государственный технологический университет",
                                     "БГТУ",
                                     "г. Минск, Свердлова, 13А");
    Faculty fit = new Faculty("Информационных технологий", "ИТ", "г. Минск, Свердлова, 13А, 101-4");
    Faculty fie = new Faculty("Информационных технологий", "ИТ", "г. Минск, Свердлова, 13А, 304-4");
    Faculty flh = new Faculty("Лекции", "Л", "г. Минск, Свердлова, 13А, 210-1");

    bstu.addFaculty(fit);
    bstu.addFaculty(fie);
    bstu.addFaculty(flh);

    Organization org_bstu = bstu;
    Organization org_fit = fit;
    Organization org = new Organization("Стадион Динамо", "Динамо", "г. Минск, ул. Ульяновская, 8");

    IStaff x = new Faculty("Принттехнологий и медиакоммуникаций", "ПМ", "г. Минск, Свердлова, 13А, 150-4");

    List<JobVacancy> list = Process.mergeJobVacancies(bstu, fit, fie, flh, org);
    Console.WriteLine(Process.printJobVacancies(list));
    Console.WriteLine("-----");

    Console.WriteLine(org_bstu.PrintInfo());
    Console.WriteLine(org_fit.PrintInfo());
    Console.WriteLine(org.PrintInfo());
    Console.WriteLine(((IStaff)org).printJobVacancies());
    Console.WriteLine(((IStaff)bstu).printJobVacancies());
    Console.WriteLine(((IStaff)fit).printJobVacancies());
    Console.WriteLine(((IStaff)fie).printJobVacancies());
    Console.WriteLine(((IStaff)org_fit).printJobVacancies());
    Console.WriteLine(x.printJobVacancies());
}
```

43. КОНЕЦ